

The Temporal Logic Synthesis Format TLSF v1.2

Swen Jacobs

CISPA Helmholtz Center for Information Security
Saarbrücken, Germany
jacobs@cispa.de

Guillermo A. Pérez

University of Antwerp – Flanders Make
Antwerp, Belgium
guillermo.perez@uantwerpen.be

Philipp Schlehuber-Caissier

EPITA Research Laboratory
Paris, France
philipp@lrde.epita.fr

We present an extension of the Temporal Logic Synthesis Format (TLSF). TLSF builds on standard LTL, but additionally supports high-level constructs, such as sets and functions, as well as parameters that allow a specification to define a whole a family of problems. Our extension introduces operators and a new semantics option for $\mathbf{LTL}_f$, i.e., LTL on finite executions.

We assume the reader is already familiar with the TLSF (v1.1) synthesis format, please refer to [5].

1 The Basic Format

We first recap the basic format of TLSF v1.1 in sections Sections 1.1 to 1.3, and then introduce extensions for $\mathbf{LTL}_f$ in Section 1.4.

A specification in the basic format consists of an **INFO** section and a **MAIN** section:

$\langle info \rangle \langle main \rangle$

1.1 The INFO Section

The **INFO** section contains the meta data of the specification, like a title and some description¹. Furthermore, it defines the underlying semantics of the specification (Mealy or Moore / standard, strict implication, or finite semantics) and the target model of the synthesized implementation. Detailed information about supported semantics and targets can be found in Sect. 3. Finally, a comma separated list of tags can be specified to identify features of the specification, e.g., the restriction to a specific fragment of LTL. A $\langle tag \rangle$ can be any string literal and is not restricted to any predefined keywords.

```
INFO {
  TITLE:      "some title"
  DESCRIPTION: "some description"
  SEMANTICS:  semantics
  TARGET:     target
  TAGS:       tag, tag, ...
}
```

¹We use colored verbatim font to identify the syntactic elements of the specification.

1.2 The MAIN Section

The specification is completed by the **MAIN** section, which contains the partitioning of input and output signals, followed by the main specification. The specification itself is separated into assumptions on the environment and desired properties of the system, and can additionally be distinguished into initial (**INITIALLY/PRESET**), invariant (**REQUIRE/ASSERT**), and arbitrary (**ASSUME/GUARANTEE**) properties². Multiple declarations and expressions need to be separated by a `;`.

```

MAIN {
  INPUTS    { (⟨boolean signal declaration⟩)* }
  OUTPUTS   { (⟨boolean signal declaration⟩)* }
  INITIALLY { (⟨basic LTL expression⟩)* }
  PRESET    { (⟨basic LTL expression⟩)* }
  REQUIRE   { (⟨basic LTL expression⟩)* }
  ASSERT    { (⟨basic LTL expression⟩)* }
  ASSUME    { (⟨basic LTL expression⟩)* }
  GUARANTEE { (⟨basic LTL expression⟩)* }
}

```

All subsections except **INPUTS** and **OUTPUTS** are optional.

1.3 Basic Expressions

A basic expression e is either a boolean signal or a basic LTL expression. Each basic expression has a corresponding type that is $\mathbb{S}$ for boolean signals and $\mathbb{T}$ for LTL expressions. Basic expressions can be composed to larger expressions using operators. An overview over the different types of expressions and operators is given below.

Boolean Signal Declarations. A signal identifier is represented by a string consisting of lowercase and uppercase letters (`'a'-'z', 'A'-'Z'`), numbers (`'0'-'9'`), underscores (`'_'`), primes (`' '`), and at-signs (`'@'`) and does not start with a number or a prime. Additionally, keywords like **X**, **G** or **U**, as defined in the rest of this document, are forbidden. An identifier is declared as either an input or an output signal. We denote the set of declared input signals as $\mathcal{I}$ and the set of declared output signals as $\mathcal{O}$, where $\mathcal{I} \cap \mathcal{O} = \emptyset$. Then, a boolean signal declaration simply consists of a signal identifier $\langle name \rangle$ from $\mathcal{I} \cup \mathcal{O}$.

Basic LTL Expressions. A basic LTL expression conforms to the following grammar, including truth values, signals, boolean operators and temporal operators. For easy parsing of the basic format, we require fully parenthesized expressions, as expressed by the first of the following lines:

$$\begin{aligned}
 \varphi &\equiv (\varphi') \\
 \varphi' &\equiv \text{true} \mid \text{false} \mid s \text{ for } s \in \mathcal{I} \cup \mathcal{O} \mid \\
 &\quad !\varphi \mid \varphi \ \&\& \ \varphi \mid \varphi \ \|\ \varphi \mid \varphi \rightarrow \varphi \mid \varphi \leftrightarrow \varphi \mid \\
 &\quad \text{X } \varphi \mid \text{G } \varphi \mid \text{F } \varphi \mid \varphi \text{ U } \varphi \mid \varphi \text{ R } \varphi \mid \varphi \text{ W } \varphi
 \end{aligned}$$

²In TLSF v1.0 [4], **ASSERT** was called **INVARIANTS**, **ASSUME** was called **ASSUMPTIONS**, and **GUARANTEE** was called **GUARANTEES** (and subsections **INITIALLY**, **PRESET**, and **REQUIRE** did not exist). TLSF v1.1 still supports the old identifiers.

Thus, a basic LTL expression is either true, false, or a signal, or composed from these atomic expressions with boolean operators (negation, conjunction, disjunction, implication, equivalence) and temporal operators (next, globally, eventually, until, release, weak until). The semantics of the boolean and temporal operators are defined in the usual way.

1.4 New Basic Expressions for $\mathbf{LTL}_f$

A basic $\mathbf{LTL}_f$ expression conforms to the following grammar. As before, we require fully parenthesized expressions in the basic format:

$$\begin{aligned} \varphi &\equiv (\varphi') \\ \varphi' &\equiv \text{true} \mid \text{false} \mid s \text{ for } s \in \mathcal{I} \cup \mathcal{O} \mid \\ &\quad !\varphi \mid \varphi \ \&\& \ \varphi \mid \varphi \ \parallel \ \varphi \mid \varphi \rightarrow \varphi \mid \varphi \leftrightarrow \varphi \mid \\ &\quad \mathbf{X} \varphi \mid \mathbf{G} \varphi \mid \mathbf{F} \varphi \mid \varphi \ \mathbf{U} \ \varphi \mid \varphi \ \mathbf{R} \ \varphi \mid \varphi \ \mathbf{W} \ \varphi \mid \\ &\quad \mathbf{X}[\!] \varphi \mid \end{aligned}$$

Thus, a basic $\mathbf{LTL}_f$ expression can contain the same operators as an LTL expression, but additionally may contain the *strong next* operator $\mathbf{X}[\!]$. Also note that $\mathbf{X}$ is interpreted as *weak next* in $\mathbf{LTL}_f$, as described in Section 2.

2 LTL over Finite Words

We now recall the semantics of $\mathbf{LTL}_f$ over finite words as well as the synthesis problem for $\mathbf{LTL}_f$ specifications. As is clear from the previous section, the syntax of $\mathbf{LTL}_f$ is very similar to that of LTL. In fact, syntactically it only adds a *strong next* operator.

For the rest of this section, we write AP to denote the set $\mathcal{I} \cup \mathcal{O}$ of all atomic propositions.

2.1 Semantics over Finite Words

Now, the semantics of an $\mathbf{LTL}_f$ formula φ over finite words $\alpha = a_1 a_2 \dots a_n \in (2^{AP})^+$ of lengths $n > 0$ are inductively defined $\forall i$ s.t. $0 < i \leq n$ as follows.

- $\alpha, i \models \text{true}$
- $\alpha, i \models p$ iff $p \in a_i$
- $\alpha, i \models !\varphi$ iff $\alpha, i \not\models \varphi$
- $\alpha, i \models \varphi \parallel \psi$ iff $\alpha, i \models \varphi$ or $\alpha, i \models \psi$
- $\alpha, i \models \mathbf{X}\varphi$ iff $i = n$ or $\alpha, i + 1 \models \varphi$
- $\alpha, i \models \mathbf{X}[\!]\varphi$ iff $i < n$ and $\alpha, i + 1 \models \varphi$
- $\alpha, i \models \mathbf{F}\varphi$ iff $\exists j$ s.t. $i \leq j \leq n$ we have $\alpha, j \models \varphi$
- $\alpha, i \models \mathbf{G}\varphi$ iff $\forall j$ s.t. $i \leq j \leq n$ we have $\alpha, j \models \varphi$
- $\alpha, i \models \varphi_1 \ \mathbf{U} \ \varphi_2$ iff $\exists k$ s.t. $i \leq k \leq n$ and $\alpha, k \models \varphi_2$; additionally, $\forall j$ s.t. $\forall i \leq j < k$, we have $\alpha, j \models \varphi_1$
- $\alpha, i \models \varphi_1 \ \mathbf{W} \ \varphi_2$ iff $\varphi_1 \ \mathbf{U} \ \varphi_2$ or $\mathbf{G}\varphi_1$

Finally, we say α satisfies φ , written $\alpha \models \varphi$, if $\alpha, 0 \models \varphi$ holds. We adopt the convention that $\varepsilon \not\models \varphi$, for any $\mathbf{LTL}_f$ formula φ . In words: the empty word does not satisfy any $\mathbf{LTL}_f$ formula.

Note the asymmetry between the two versions of the next operator. The strong version requires that there be a next letter in the word.

2.2 Synthesis and realizability

Notice that there is a natural bijection between words in $(2^{AP})^*$ and those in $(2^{\mathcal{I}} \times 2^{\mathcal{O}})^*$. For convenience, we work with words of the latter type while still speaking of them satisfying $\mathbf{LTL}_f$ formulas over AP via said bijection. Additionally, we shall write $I = 2^{\mathcal{I}}$ and $O = 2^{\mathcal{O}}$ to avoid heavy notation.

We say a function $f : I^* \times I \rightarrow O$ is a *Mealy controller*, while $f : I^* \rightarrow O$ is a *Moore controller*. Now, for a (potentially infinite) word $\alpha = (a_0, b_0) \dots$, with $(a_i, b_i) \in I \times O$ for all $i \geq 0$, we say α is consistent with a controller f if f is a Mealy controller (respectively Moore) and $b_i = f(a_0 \dots a_{i-1}, a_i)$ (resp. $b_i = f(a_0 \dots a_{i-1})$) for all $i \geq 0$. This notation allows us to speak about *realizability*: an $\mathbf{LTL}_f$ specification φ is realizable if there exists a controller f such that all words $\alpha \in (I \times O)^\omega$ consistent with it have some prefix that satisfies φ . The *synthesis* problem asks to construct, given a specification φ , a controller of the specified type witnessing the realizability of φ if one exists.

We refer the reader to [3] and references therein for more information about the history and the computational complexity of the synthesis problem for specifications in $\mathbf{LTL}_f$.

3 Targets and Semantics

3.1 Targets

The **TARGET** of the specification defines the implementation model that a solution should adhere to. Currently supported targets are Mealy automata (**Mealy**), whose output depends on the current state and input, and Moore automata (**Moore**), whose output only depends on the current state. The differentiation is necessary since realizability of a specification depends on the target system model. For example, every specification that is realizable under Moore semantics is also realizable under Mealy semantics, but not vice versa.

3.2 Semantics

The **SEMANTICS** of the specification defines how the formula was intended to be evaluated, which also depends on an implementation model. We now support six different semantics, which are standard Mealy semantics (**Mealy**) and standard Moore semantics (**Moore**), as well as their strict and finite variants (**TARGET**, **VARIANT**), where **VARIANT** $\in \{\mathbf{Strict}, \mathbf{Finite}\}$.

In the following, consider a specification where **INITIALLY** evaluates to the LTL formula θ_e , **PRESET** evaluates to θ_s , **REQUIRE** evaluates to ψ_e , **ASSERT** evaluates to ψ_s , **ASSUME** evaluates to φ_e , and **GUARANTEE** evaluates to φ_s . For specification sections that are not present, the respective formula is interpreted as **true**.

Standard semantics. If the semantics is (standard) **Mealy** or **Moore**, and the **TARGET** coincides with the semantics system model, then the specification is interpreted as the formula

$$\theta_e \rightarrow (\theta_s \wedge (G \psi_e \wedge \varphi_e \rightarrow G \psi_s \wedge \varphi_s)) \quad (1)$$

in standard LTL semantics. Note that we require that the **PRESET** property θ_s holds whenever the **INITIALLY** condition θ_e holds, regardless of other environment assumptions.

Strict semantics. If the semantics is **Mealy,Strict** or **Moore,Strict**, and the **TARGET** coincides with the semantics system model, then the specification is interpreted under strict implication semantics (as used in the synthesis of GR(1) specifications), which is equivalent to the formula

$$\theta_e \rightarrow (\theta_s \wedge (\psi_s \mathcal{W} \neg \psi_e) \wedge (G \psi_e \wedge \varphi_e \rightarrow \varphi_s))$$

in standard LTL semantics. In this case, we additionally require that the **ASSERT** property ψ_s needs to hold at least as long as the **REQUIRE** condition ψ_e holds.

Note that this gives us an easy way to convert a specification with strict semantics into one with non-strict semantics. For details on strict implication semantics, see Klein and Pnueli [6], as well as Bloem et al. [1], from which we also take our definition and interpretation of the GR(1) fragment.³

Finite Semantics. If the semantics is (finite) **Mealy,Finite** or **Moore,Finite**, and the **TARGET** coincides with the semantics system model, then the specification is interpreted like in the standard semantics, i.e. as the formula in Equation (1). However, this time the formula is to be interpreted in the **LTL_f** semantics (see Section 2).

4 The Full Format

In the full format, a specification consists of three sections: the **INFO** section, the **GLOBAL** section and the **MAIN** section. The **GLOBAL** section is optional.

$$\langle info \rangle [\langle global \rangle] \langle main \rangle$$

The **INFO** section is the same as in the basic format, defined in Sect. 1.1. The **GLOBAL** section can be used to define parameters, and to bind identifiers to expressions that can be used later in the specification. The **MAIN** section is used as before, but can use extended sets of declarations and expressions.

We define the **GLOBAL** section in Sect. 4.1, and the changes to the **MAIN** section compared to the basic format in Sect. 4.2. The extended set of expressions that can be used in the full format is introduced in Sect. 4.3, enumerations, extended signal and function declarations in Sect. 4.4 and 4.6, and additional notation in Sect. 4.7–4.9.

4.1 The GLOBAL Section

The **GLOBAL** section consists of the **PARAMETERS** subsection, defining the identifiers that parameterize the specification, and the **DEFINITIONS** subsection, that allows to define functions, enumerations and to bind identifiers to complex expressions. Multiple declarations need to be separated by a **;**. The section and its subsections are optional.

```
GLOBAL {
  PARAMETERS {
    (identifier) = (numerical expression);
```

³Note that in the conversion of [1], the formula is strengthened by adding the formula $G(H\psi_e \rightarrow \psi_s)$, where $H\varphi$ is a Past-LTL formula and denotes that φ holds everywhere in the past. However, it is easy to show that our definition of strict semantics matches the definition of [1]. We prefer this notion, since it avoids the introduction of Past-LTL.

```

}
DEFINITIONS {
  (((function declaration) | (enum declaration) | (identifier) = (expression));)*
}
}

```

4.2 The MAIN Section

Like in the basic format, the **MAIN** section contains the partitioning of input and output signals, as well as the main specification. However, signal declarations can now contain signal buses, and LTL expressions can use parameters, functions, and identifiers defined in the **GLOBAL** section.

```

MAIN {
  INPUTS    { (signal declaration);* }
  OUTPUTS   { (signal declaration);* }
  INITIALLY { (LTL expression);* }
  PRESET    { (LTL expression);* }
  REQUIRE   { (LTL expression);* }
  ASSERT    { (LTL expression);* }
  ASSUME    { (LTL expression);* }
  GUARANTEE { (LTL expression);* }
}

```

As before, all subsections except **INPUTS** and **OUTPUTS** are optional.

4.3 Expressions

An expression e is either a boolean signal, an n -ary signal (called bus), an enumeration type, a numerical expression, a boolean expression, an LTL expression, or a set expression. Each expression has a corresponding type that is either one of the basic types: $\mathbb{S}, \mathbb{U}, \mathbb{E}, \mathbb{N}, \mathbb{B}, \mathbb{T}$, or a recursively defined set type $S_{\mathbb{X}}$ for some type $\mathbb{X}$.

As before, an identifier is represented by a string consisting of lowercase and uppercase letters ('a'-'z', 'A'-'Z'), numbers ('0'-'9'), underscores ('_'), primes (''), and at-signs ('@') and does not start with a number or a prime. In the full format, identifiers are bound to expressions of different type. We denote the respective sets of identifiers by $\Gamma_{\mathbb{S}}, \Gamma_{\mathbb{U}}, \Gamma_{\mathbb{E}}, \Gamma_{\mathbb{N}}, \Gamma_{\mathbb{B}}, \Gamma_{\mathbb{T}}$, and $\Gamma_{S_{\mathbb{X}}}$. Finally, basic expressions can be composed to larger expressions using operators. In the full format, we do not require fully parenthesized expressions. If an expression is not fully parenthesized, we use the precedence order given in Table 1. An overview over the all types of expressions and operators is given below.

Numerical Expressions. A numerical expression $e_{\mathbb{N}}$ conforms to the following grammar:

$$\begin{aligned}
 e_{\mathbb{N}} \equiv & i \text{ for } i \in \Gamma_{\mathbb{N}} \mid n \text{ for } n \in \mathbb{N} \mid e_{\mathbb{N}} + e_{\mathbb{N}} \mid e_{\mathbb{N}} - e_{\mathbb{N}} \mid e_{\mathbb{N}} * e_{\mathbb{N}} \mid e_{\mathbb{N}} / e_{\mathbb{N}} \mid e_{\mathbb{N}} \% e_{\mathbb{N}} \\
 & \mid e_{S_{\mathbb{X}}} \mid \text{MIN } e_{S_{\mathbb{N}}} \mid \text{MAX } e_{S_{\mathbb{N}}} \mid \text{SIZEOF } s \text{ for } s \in \Gamma_{\mathbb{U}}
 \end{aligned}$$

Thus, a numerical expression either represents an identifier (bound to a numerical value), a numerical constant, an addition, a subtraction, a multiplication, an integer division, a modulo operation, the size

Precedence	Operator	Description	Arity	Associativity
1	+ [.] (SUM[.]) * [.] (PROD[.]) ... (SIZE) MIN MAX SIZEOF	sum product size minimum maximum size of a bus	unary	
2	* (MUL)	multiplication	binary	left-to-right
3	/ (DIV) % (MOD)	integer division modulo	binary	right-to-left
4	+ (PLUS) - (MINUS)	addition difference	binary	left-to-right
5	(*) [.] (CAP[.]) (+) [.] (CUP[.])	intersection union	unary	
6	(\) ((-), SETMINUS)	set difference	binary	right-to-left
7	(*) (CAP)	intersection	binary	left-to-right
8	(+) (CUP)	union	binary	left-to-right
9	== (EQ) != (/=, NEQ) < (LE) <= (LEQ) > (GE) >= (GEG)	equality inequality smaller than smaller or equal than greater then greater or equal than	binary	left-to-right
10	IN (ELEM, <-)	membership	binary	left-to-right
11	! (NOT) X X[!] F G && [.] (AND[.], FORALL[.]) [.] (OR[.], EXISTS[.])	negation next strong next finally globally conjunction disjunction	unary	
12	&& (AND)	conjunction	binary	left-to-right
13	(OR)	disjunction	binary	left-to-right
14	-> (IMPLIES) <-> (EQUIV)	implication equivalence	binary	right-to-left
15	W	weak until	binary	right-to-left
16	U	until	binary	right-to-left
17	R	release	binary	left-to-right
18	~	pattern match	binary	left-to-right
19	:	guard	binary	left-to-right

Table 1: The table lists the precedence, arity and associativity of all expression operators. Also consider the alternative names in brackets which can be used instead of the symbolic representations.

of a set, the minimal/maximal value of a set of naturals, or the size (i.e., width) of a bus, respectively. The semantics are defined in the usual way.

Set Expressions. A set expression $e_{\mathbb{X}}$, containing elements of type $\mathbb{X}$, conforms to the following grammar:

$$e_{\mathbb{X}} \equiv i \text{ for } i \in \Gamma_{\mathbb{S}_{\mathbb{X}}} \mid \{e_{\mathbb{X}}, e_{\mathbb{X}}, \dots, e_{\mathbb{X}}\} \mid \{e_{\mathbb{N}}, e_{\mathbb{N}} \dots e_{\mathbb{N}}\} \mid \\ e_{\mathbb{S}_{\mathbb{X}}} (+) e_{\mathbb{S}_{\mathbb{X}}} \mid e_{\mathbb{S}_{\mathbb{X}}} (*) e_{\mathbb{S}_{\mathbb{X}}} \mid e_{\mathbb{S}_{\mathbb{X}}} (\setminus) e_{\mathbb{S}_{\mathbb{X}}}$$

Thus, the expression $e_{\mathbb{S}_{\mathbb{X}}}$ either represents an identifier (bound to a set of values of type $\mathbb{X}$), an explicit list of elements of type $\mathbb{X}$, a list of elements specified by a range (for $\mathbb{X} = \mathbb{N}$), a union of two sets, an intersection or a difference, respectively. The semantics of a range expression $\{x, y \dots z\}$ are defined for $x < y$ via:

$$\{n \in \mathbb{N} \mid x \leq n \leq z \wedge \exists j. n = x + j \cdot (y - x)\}.$$

The semantics of all other expressions are defined as usual. Sets contain either positive integers, boolean expressions, LTL expressions, buses, signals, or other sets of a specific type.

Boolean Expressions. A boolean expression $e_{\mathbb{B}}$ conforms to the following grammar:

$$e_{\mathbb{B}} \equiv i \text{ for } i \in \Gamma_{\mathbb{B}} \mid e_{\mathbb{X}} \text{ IN } e_{\mathbb{S}_{\mathbb{X}}} \mid \text{true} \mid \text{false} \mid !e_{\mathbb{B}} \mid \\ e_{\mathbb{B}} \&\& e_{\mathbb{B}} \mid e_{\mathbb{B}} \mid\mid e_{\mathbb{B}} \mid e_{\mathbb{B}} \rightarrow e_{\mathbb{B}} \mid e_{\mathbb{B}} \leftrightarrow e_{\mathbb{B}} \mid \\ e_{\mathbb{N}} == e_{\mathbb{N}} \mid e_{\mathbb{N}} != e_{\mathbb{N}} \mid e_{\mathbb{N}} < e_{\mathbb{N}} \mid e_{\mathbb{N}} <= e_{\mathbb{N}} \mid e_{\mathbb{N}} > e_{\mathbb{N}} \mid e_{\mathbb{N}} >= e_{\mathbb{N}}$$

Thus, a boolean expression either represents an identifier (bound to a boolean value), a membership test, true, false, a negation, a conjunction, a disjunction, an implication, an equivalence, or an equation between two positive integers (equality, inequality, less than, less or equal than, greater than, greater or equal than), respectively. The semantics are defined in the usual way. Note that signals are not allowed in a boolean expression, but only in an LTL expression.

LTL Expressions. An LTL expression φ conforms to the same grammar as a boolean expression, except that it additionally includes signals and temporal operators.

$$\varphi \equiv \dots \mid i \text{ for } i \in \Gamma_{\mathbb{T}} \mid s \text{ for } s \in \Gamma_{\mathbb{S}} \mid b[e_{\mathbb{N}}] \text{ for } b \in \Gamma_{\mathbb{U}} \mid \\ b_0 == b_1 \text{ for } b_j \in \Gamma_{\mathbb{U}} \text{ and } b_{1-j} \in \Gamma_{\mathbb{E}} \mid b_0 != b_1 \text{ for } b_j \in \Gamma_{\mathbb{U}} \text{ and } b_{1-j} \in \Gamma_{\mathbb{E}} \mid \\ \text{X } \varphi \mid \text{X}[\text{!}] \varphi \mid \text{G } \varphi \mid \text{F } \varphi \mid \varphi \text{ U } \varphi \mid \varphi \text{ R } \varphi \mid \varphi \text{ W } \varphi$$

Thus, an LTL expression additionally can represent an identifier bound to an LTL formula, a signal, an $e_{\mathbb{N}}$ -th signal of a bus, a next operation, a restriction of a bus to a set of enumeration valuations via equality or inequality, a globally operation, an eventually operation, an until operation, a release operation, or a weak until operation, respectively. Note that every boolean expression is also an LTL expression, thus we allow the use of identifiers that are bound to boolean expressions as well. The semantics of expressions involving bus operations is defined in the subsequent sections.

4.4 Enumerations

An enumeration declaration conforms to the following grammar:

$$\text{enum } \langle \text{enumtype} \rangle = \left(\langle \text{identifier} \rangle : (0 \mid 1 \mid *)^n (, (0 \mid 1 \mid *)^n)^* \right)^+$$

for some arbitrary but fix positive integer $n > 0$. As an example consider the enumeration `Positions`, which declares the enumeration identifiers `LEFT`, `MIDDLE`, `RIGHT`, and `UNDEF` as members of $\Gamma_{\mathbb{E}}$:

```
enum Position =
  LEFT: 100
  MIDDLE: 010
  RIGHT: 001
  UNDEF: 11*, 1*1, *11
```

We use `0` to identify the absent signal, `1` to identify the present signal and `*` for either of both. Each identifier then refers to at least one concrete signal valuation sequence. Multiple values can be denoted by sequences with a `*`, as well as by comma separated lists. Furthermore, the identifier of each declared valuation has to be unique. Not all possible valuations have to be identified.

Enumeration identifiers can only be used in comparisons against buses inside an LTL expression, where we require that the corresponding bus has the same width as the valuation compared to. It defines a boolean constraint on the bus, restricting it to the different valuations, bound to the identifier, e.g., the expressions `b == RIGHT` and `!b[0] && !b[1] && b[2]` are semantically equivalent, as well as `b /= UNDEF` and `!(b[0] && b[1]) || (b[0] && b[2]) || (b[1] && b[2])`.

4.5 Signals and Buses

A single signal declaration consists of the name of the signal. As for the basic format, signals are declared as either input or output signals, denoted by $\mathcal{I}$ and $\mathcal{O}$, respectively. A bus declaration additionally specifies a signal width, i.e., a bus represents a finite set of signals. The signal width is either given by a numerical value or via an enumeration type.

$$\langle name \rangle \mid \langle name \rangle [e_{\mathbb{N}}] \mid \langle enumtype \rangle \langle name \rangle$$

Semantically, a signal declaration `s` specifies a signal $s \in \mathcal{I} \cup \mathcal{O}$, where a bus declaration `b[n]` specifies n signals `b[0]`, `b[1]`, ..., `b[n-1]`, with either `b[i] ∈ I` for all $0 \leq i < n$, or `b[i] ∈ O` for all $0 \leq i < n$. A bus specified via an enumeration type has the same width as the valuations of the corresponding enumeration.

Buses which are declared using an enumeration type, where not all valuations are related to an identifier⁴ induce an implicit constraint on the corresponding signals: if the bus corresponds to a set of input signals, then the global requirement that no other than the defined valuations appear on this bus is imposed. If it corresponds to a set of output signals, then the equivalent global invariant is imposed.

Finally, note that we use `b[i]` to access the i -th value of b , i.e., we use the same syntax as for the declaration itself⁵. Also note that for the declared signals s , we have $s \in \mathcal{I} \cup \mathcal{O} \subseteq \Gamma_{\mathbb{S}}$, and for the declared buses b , we have $b \in \Gamma_{\mathbb{U}}$.

4.6 Function Declarations

As another feature, one can declare (recursive) functions of arbitrary arity inside the `DEFINITIONS` section. Functions can be used to define simple macros, but also to generate complex formulas from a

⁴See e.g. the `000` valuation of the example of Sect. 4.5

⁵C-Array Syntax Style

given set of parameters. A declaration of a function of arity n has the form

$$\langle \text{function name} \rangle (\langle \text{arg}_1 \rangle, \langle \text{arg}_2 \rangle, \dots, \langle \text{arg}_n \rangle) = (e_c)^+,$$

where $\langle \text{arg}_1 \rangle, \langle \text{arg}_2 \rangle, \dots, \langle \text{arg}_n \rangle$ are fresh identifiers that can only be used inside the sub-expressions e_c . An expression e_c conforms to the following grammar:

$$e_c \equiv e \mid e_{\mathbb{B}} : e \mid e_{\mathbb{P}} : e \quad \text{where } e \equiv e_{\mathbb{N}} \mid e_{\mathbb{B}} \mid e_{S_{\mathbb{X}}} \mid \varphi$$

Thus, a function can be bound to any expression e , parameterized in its arguments, which additionally may be guarded by some boolean expression $e_{\mathbb{B}}$, or a pattern match $e_{\mathbb{P}}$. If the regular expression $(e_c)^+$ consists of more than one expression e_c , then the function binds to the first expression whose guard evaluates to **true** (in the order of their declaration). Furthermore, the special guard **otherwise** can be used, which evaluates to **true** if and only if all other guards evaluate to **false**. Expressions without a guard are implicitly guarded by **true**. All sub-expressions e_c need to have the same type $\mathbb{X}$. For every instantiation of a function by given parameters, we view the resulting expression $e_{\mathbb{X}}$ as an identifier in $\Gamma_{\mathbb{X}}$, bound to the result of the function application.

Pattern Matching. Pattern matches are special guards of the form

$$e_{\mathbb{P}} \equiv \varphi \sim \varphi',$$

which can be used to describe different behavior depending on the structure of an LTL expression. Hence, a guard $e_{\mathbb{P}}$ evaluates to **true** if and only if φ and φ' are structurally equivalent, with respect to their boolean and temporal connectives. Furthermore, identifier names that are used in φ' need to be fresh, since every identifier expression that appears in φ' is bound to the equivalent sub-expression in φ , which is only visible inside the right-hand-side of the guard. Furthermore, to improve readability, the special identifier `_` (wildcard) can be used, which always remains unbound. To clarify this feature, consider the following function declaration:

```
fun(f) =
  f ~ a U _ : a
  otherwise: X f
```

The function *fun* gets an LTL formula f as a parameter. If f is an until formula of the form $\varphi_1 \mathcal{U} \varphi_2$, then *fun*(f) binds to φ_1 , otherwise *fun*(f) binds to $X f$.

4.7 Big Operator Notation

It is often useful to express parameterized expressions using “big” operators, e.g., we use Σ to denote a sum over multiple sub-expressions, Π to denote a product, or $\bigcup$ to denote a union. It is also possible to use this kind of notion in this specification format. The corresponding syntax looks as follows:

$$\langle op \rangle [\langle id_0 \rangle \text{IN } e_{S_{\mathbb{X}_0}}, \langle id_1 \rangle \text{IN } e_{S_{\mathbb{X}_1}}, \dots, \langle id_n \rangle \text{IN } e_{S_{\mathbb{X}_n}}] e_{\mathbb{X}}$$

Let x_j be the identifier represented by $\langle id_j \rangle$ and S_j be the set represented by $e_{S_{\mathbb{X}_j}}$. Further, let $\bigoplus$ be the mathematical operator corresponding to $\langle op \rangle$. Then, the above expression corresponds to the mathematical expression:

$$\bigoplus_{x_0 \in S_0} \bigoplus_{x_1 \in S_1} \dots \bigoplus_{x_n \in S_n} (e_{\mathbb{X}})$$

Note that $\langle id_0 \rangle$ is already bound in expression $e_{S_{\mathbb{X}_1}}$, $\langle id_1 \rangle$ is bound in $e_{S_{\mathbb{X}_2}}$, and so forth. The syntax is supported by every operator $\langle op \rangle \in \{+, *, (+), (*), \&\&, ||\}$.

4.8 Syntactic Sugar

To improve readability, there is additional syntactic sugar, which can be used beside the standard syntax. Let n and m be numerical expressions, then

- $X[n] \varphi$ denotes a stack of n next operations, e.g.:

$$X[3] a \equiv X X X a$$

- $F[n:m] \varphi$ denotes that φ holds somewhere between the next n and m steps, e.g.:

$$F[2:3] a \equiv X X(a \mid \mid X a)$$

- $G[n:m] \varphi$ denotes that φ holds everywhere between the next n and m steps, e.g.:

$$G[1:3] a \equiv X(a \ \&\& \ X(a \ \&\& \ X a))$$

- All of the above have been overloaded so that, with the finite semantics, they use the strong next operator if a $!$ appears inside (i.e. at the beginning or the end) the square braces. For instance:

$$G[!1:3] a \equiv G[1:3!] a \equiv X[!](a \ \&\& \ X[!](a \ \&\& \ X[!] a))$$

- $\langle op \rangle [\dots, n \circ_1 \langle id \rangle \circ_2 m, \dots] e_X$ denotes a big operator application, where $n \circ_1 \langle id \rangle \circ_2 m$ with $\circ_1, \circ_2 \in \{<, <=\}$ denotes that $\langle id \rangle$ ranges from n to m . The inclusion of n and m depends on the choice of $\circ_1$ and $\circ_2$, respectively. Thus, the notation provides an alternative to membership in combination with set ranges, e.g.:

$$\&\&[0 <= i < n] b[i] \equiv \&\&[i \text{ IN } \{0, 1..n-1\}] b[i]$$

4.9 Comments

It is possible to use C style comments anywhere in the specification, i.e., there are single line comments initialized by `//` and multi line comments between `/*` and `*/`. Multi line comments can be nested.

5 Updates to the SyFCo Tool

We have updated the Synthesis Format Conversion Tool⁶ (SyFCo) to support TLSF specifications over finite words. For this, we now support output in the format `ltlxba-fin`. The output format is similar to versions of `LTLf` that the Spot tool [2] can handle.

Acknowledgements

We would like to thank Luca Geatti, Nicola Gigante, Antonio Di Stasio, and Shufang Zhu for their valuable feedback on earlier versions of this note.

References

- [1] Roderick Bloem, Barbara Jobstmann, Nir Piterman, Amir Pnueli & Yaniv Sa’ar (2012): *Synthesis of Reactive(1) Designs*. *J. Comput. Syst. Sci.* 78(3), pp. 911–938, doi:10.1016/j.jcss.2011.08.007.

⁶See <https://github.com/reactive-systems/syfco>

- [2] Alexandre Duret-Lutz, Etienne Renault, Maximilien Colange, Florian Renkin, Alexandre Gbaguidi Aisse, Philipp Schlehuber-Caissier, Thomas Medioni, Antoine Martin, Jérôme Dubois, Clément Gillard & Henrich Lauko (2022): *From Spot 2.0 to Spot 2.10: What's New?* In: *Proceedings of the 34th International Conference on Computer Aided Verification (CAV'22)*, *Lecture Notes in Computer Science* 13372, Springer, pp. 174–187, doi:10.1007/978-3-031-13188-2_9.
- [3] Giuseppe De Giacomo & Moshe Y. Vardi (2015): *Synthesis for LTL and LDL on Finite Traces*. In Qiang Yang & Michael J. Wooldridge, editors: *Proceedings of the Twenty-Fourth International Joint Conference on Artificial Intelligence, IJCAI 2015, Buenos Aires, Argentina, July 25-31, 2015*, AAAI Press, pp. 1558–1564. Available at <http://ijcai.org/Abstract/15/223>.
- [4] Swen Jacobs & Felix Klein (2016): *A High-Level LTL Synthesis Format: TLSF v1.0*. CoRR abs/1601.05228. Available at <http://arxiv.org/abs/1601.05228>.
- [5] Swen Jacobs, Felix Klein & Sebastian Schirmer (2016): *A High-Level LTL Synthesis Format: TLSF v1.1*. In Ruzica Piskac & Rayna Dimitrova, editors: *Proceedings Fifth Workshop on Synthesis, SYNT@CAV 2016, Toronto, Canada, July 17-18, 2016*, *EPTCS* 229, pp. 112–132, doi:10.4204/EPTCS.229.10. Available at <https://doi.org/10.4204/EPTCS.229.10>.
- [6] Uri Klein & Amir Pnueli (2010): *Revisiting Synthesis of GR(1) Specifications*. In: *HVC 2010. Revised Selected Papers*, *LNCS* 6504, Springer, pp. 161–181, doi:10.1007/978-3-642-19583-9_16.